



USN

--	--	--	--	--	--	--	--	--	--

NMAM INSTITUTE OF TECHNOLOGY, NITTE
 Off-Campus Centre of Nitte (Deemed to be University)
Third Semester B.Tech (CBCS) Degree Examinations

December 2025

VT2002-1 – DIGITAL LOGIC DESIGN WITH VERILOG
 (For VLSI)

Duration: 3 Hours

Max. Marks: 100

Note:

Part – A: Multiple Choice Questions: Answer all **Twenty** questions in the **OMR Sheet** provided. Each question carries equal marks.

Part – B: Descriptive Answer Questions: Answer **Five** full questions choosing **Two** full questions from Unit – I & Unit – II each and **One** full question from Unit – III.

PART - A: MULTIPLE CHOICE QUESTIONS

20 Marks

- Which Verilog modeling style is used in the following code?

```

module mux4to1(output Y, input A, B, C, D, input [1:0] S);
  assign Y = (S == 2'b00) ? A :
            (S == 2'b01) ? B :
            (S == 2'b10) ? C : D;
endmodule

```

A) Structural	B) Behavioral
C) Dataflow	D) Gate-level
- In Verilog, what is the purpose of the “always” block?

A) To declare variables	B) To instantiate modules
C) To specify simulation time	D) To describe behavior using procedural statements
- Which Verilog modeling style allows for describing a combinational circuit using Boolean expressions?

A) Structural	B) Dataflow
C) Not allowed in Verilog	D) Gate-level
- Which of the following accurately describes the role of the assign statement in dataflow modeling?

A) It is used to define sequential logic	B) It is used to create continuous assignments in combinational logic
C) It is primarily used in structural modeling to connect components	D) It is used to create delay elements in the design
- @posedge means

A) Transition from x to 1	B) Transition from 0 to 1
C) Transition from x to 1, x	D) Transition from 1 to 0
- What is the primary difference between gate-level and dataflow modeling in Verilog?

A) Gate level uses always()	B) Dataflow modeling describes circuits at a functional level using Boolean equations.
C) Gate-level modeling uses behavioral constructs like always blocks.	D) Dataflow modeling is used for sequential circuits, while gate-level is used for combinational.
- Which of the following Verilog statements can be used to define the dataflow style for a 4-bit adder?

A) assign sum = A + B;	B) always @ (posedge clk) sum <= A + B;
C) always @ (posedge clk) sum = A + B;	D) initial begin sum = A + B; end



8. module comparator(input [3:0] A, B, output reg greater, equal, less);

```
always @(*) begin
    if (A > B) begin
        greater = 1;
        equal = 0;
        less = 0;
    end else if (A == B) begin
        greater = 0;
        equal = 1;
        less = 0;
    end else begin
        greater = 0;
        equal = 0;
        less = 1;
    end
end
endmodule
```

What will be the output for A = 4'b1000 and B = 4'b00010?

- A) greater = 1, equal = 0, less = 0 B) greater = 0, equal = 0, less = 1
 C) greater = 0, equal = 1, less = 0 D) greater = 0, equal = 0, less = 0
9. Which of the following circuits is not combinational?
 A) Multiplexer B) Decoder
 C) Flip-Flop D) Adder
10. In the following Verilog code, what will the output Q be when clk is high and D is 1?

```
always@(posedge clk) begin
    Q <= D;
end
```

- A) 0 B) 1
 C) Undefined D) Last value of Q
11. How many flip flops will be synthesized by this code ?

```
always@(posedge clock)
begin
    q1=d;
    q2=q1;
    q3=q2;
end
```

- A) 0 B) 2
 C) 1 D) 3
12. What does the following Verilog code describe? `always @(posedge clk) q <= d;`
 A) A D latch with level-sensitive control B) An asynchronous reset for a flip-flop
 C) A D flip-flop with edge-triggered behavior D) A counter with synchronous reset
13. What does the Verilog code `always @(posedge clk) q <= (d & enable) ? d : q; model?`
 A) A D flip-flop with synchronous load and enable control B) A D latch with asynchronous reset
 C) A D flip-flop with asynchronous reset D) A shift register with enable control
14. How is a 4-bit shift register with parallel input and parallel output defined in Verilog?
 A) `reg [3:0] shift_reg; always @(posedge clk) shift_reg <= data_in;` B) `reg [3:0] shift_reg; always @(posedge clk) shift_reg <= {shift_reg[2:0], data_in};`
 C) `reg [3:0] shift_reg; always @* shift_reg = data_in;` D) `reg [3:0] shift_reg; always @(posedge clk) shift_reg <= {data_in, shift_reg[3:1]};`
15. Which type of counter changes its state on every clock pulse?
 A) Synchronous counter B) Up Counter
 C) Asynchronous counter D) Down Counter
16. What does `reg [3:0] ram [7:0];` define in Verilog?
 A) A 4-bit wide, 8-location RAM B) A 3-bit wide, 7-location RAM
 C) A 8-bit wide, 8-location RAM D) An 8-bit wide, 16-location RAM

Statement 1: Mealy machine reacts faster to inputs. **Statement 2:** Moore machine has more circuit delays. Choose the correct option:

- A) Statement 1 is true, and Statement 2 is false B) Statement 1 is true, and Statement 2 is true
 C) Statement 1 is false, and Statement 2 is true D) Statement 1 is false, and Statement 2 is false
18. **FSM stands for**
 A) Final State Machine B) Finite State Machine
 C) Finite Set Machine D) Final Set Machine
19. **Why is a Verilog testbench used?**
 A) To synthesize hardware designs B) To debug software programs
 C) To simulate and verify hardware designs D) To measure clock frequency
20. **To sensitize a path through an input of an _____ gate, all other inputs must be set to 1.**
 A) AND B) NOR
 C) OR D) EXOR

PART - B: DESCRIPTIVE ANSWER QUESTIONS

- | | Unit – I | Marks | BT* | CO* | PO* |
|--|--|-------|-----|-----|-----|
| 1. a) Write a Verilog module named bcd_to_e3_coder using behavioral modeling to translate a 4-bit Binary-Coded Decimal input (BCD_In) into its corresponding 4-bit Excess-3 code output (E3_Out). The core conversion logic must utilize a case statement to explicitly define the output for each valid BCD input (0000 through 1001). | | 8 | L*2 | 1 | 1 |
| b) For the Verilog modules shown in Fig. 1(b)
a. Comment on the coding styles
b. Draw the circuit that represents the code description. | <pre> module example4_22 (input x1,x2,x3,x4,output f); assign f=(~x1 & ~x2 & x3 & ~x4) (~x1 & ~x2 & x3 & x4) (x1 & ~x2 & ~x3 & x4) (x1 & ~x2 & x3 & ~x4) (x1 & ~x2 & x3 & x4) (x1 & x2 & ~x3 & x4); endmodule module example4_23 (input x1,x2,x3,x4,output reg f); always @ (x1, x2, x3, x4) begin f = (~x2 & x3) (x1 & ~x3 & x4); end end </pre> | 8 | L2 | 2 | 1 |
| 2. a) Write a Verilog module named binary_adder_core using behavioral modelling to implement a single-bit full adder. The module should accept three inputs (InputA, InputB, and CarryIn) and produce two outputs: TotalSum and CarryOut. The primary logic must be described using a combination of simple if-else statements within an always block to define the combinational Sum and Carry outputs. | | 8 | L3 | 1 | 1 |
| b) Construct a Verilog module named two_bit_summing_unit using structural modeling to compute the sum of two 2-bit numbers (OpA and OpB). This unit must be built from two specific, smaller components:
A half_sum_unit sub-module (calculates sum and carry for two 1-bit inputs).
A full_sum_unit sub-module (calculates sum and carry for three 1-bit inputs by instantiating two half_sum_unit modules internally).
The top-level two_bit_summing_unit must instantiate the full_sum_unit twice to handle the LSB and MSB addition, propagating the carry correctly. The final module should output a 2-bit sum (TotalSum) and a final carry-out (Cout). | | 8 | L3 | 1 | 1 |

3. a) Implement the combinational logic for a device that performs division by a constant (3) on a 4-bit BCD input, I. The module, to be named Quo_Rem_Gen, must have a 4-bit input Data_in[3:0] (restricted to values 4'b0000 to 4'b1001 and two 2-bit outputs, Q_out[1:0] (for the quotient) and R_out[1:0] (for the remainder). The functional goal is to establish a direct, look-up table style mapping between the 10 valid input codes and the required output pairs (Q, R). Provide the complete Verilog behavioral code using a case statement to define this precise mapping.
- b) Design a Verilog module named data_selector_8x1 using dataflow modeling to implement an 8-to-1 multiplexer. The module requires eight 1-bit inputs (I[7:0]), a 3-bit selection input (Sel), and a single 1-bit output (Z). The output signal Z must be defined using a single nested conditional operator (?:) statement. Also write the corresponding testbench using for loop.

8 L3 1 1

8 L3 1 1

Unit – II

4. a) Write a Verilog module named four_bit_mode_register using behavioral modeling to implement a 4-bit shift register supporting three modes: shift-left, shift-right, and parallel load. The module requires a 4-bit data input (LoadData), a serial input (SerialData), a 2-bit control input (ModeCode), and a 4-bit output (StateQ). The control codes are:
 2'b00: No Change (Hold)
 2'b01: Shift Left (LSB receives SerialData)
 2'b10: Shift Right (MSB receives SerialData)
 2'b11: Parallel Load (LoadData is transferred to StateQ)
- b) Design a Verilog module named fast_carry_4bit_sum using dataflow modeling (continuous assign statements) to implement a 4-bit adder that explicitly calculates internal carry generate and carry propagate signals for each stage. The module should accept three inputs: FirstOperand[3:0], SecondOperand[3:0], and InitialCarry (1-bit), and produce a 4-bit output FinalResult[3:0] (the sum) and a 1-bit output OverallCarry (the final carry-out). The logic for each of the four stages (i=0 to 3) must be calculated using the following continuous assignments:
 Carry Generate (Gcreate):
 Gcreate[i]=FirstOperand[i] AND SecondOperand[i]
 Carry Propagate (Pactive):
 Pactive[i]=FirstOperand[i] OR SecondOperand[i]
 Next Stage Carry (Ci+1):
 Ci+1=Gcreate[i] OR (Pactive[i] AND Ci)
 Use these explicitly defined G and P terms along with continuous assign statements to model the full 4-bit addition, ensuring the final carry-out (OverallCarry) is properly derived.
5. a) Design a Verilog module named four_bit_bidi_counter using behavioral modeling to implement a 4-bit binary counter capable of counting both up and down. The module requires a clock input (CLK), a control signal (Direction) where '1' means count up and '0' means count down, and a 4-bit output (CountValue).

8 L3 3 1

8 L3 3 1

8 L3 4 1

- b) Construct a Verilog module named `data_selector_8x1` that realizes an 8-to-1 multiplexer using a structural cascade of smaller multiplexers. The design must specifically incorporate:

Two instances of a 4-to-1 multiplexer.

One instance of a 2-to-1 multiplexer.

The top-level `data_selector_8x1` module must have: Eight 1-bit input lines: `DataInput[7:0]` (D7 to D0). A 3-bit select input: `Selector[2:0]` (S2 being the MSB). A single 1-bit output: `ResultY`. Ensure that the most significant bit of the 3-bit select line controls the final 2-to-1 stage. Furthermore, create a separate Verilog test bench file (`mux8x1_tb`) to exhaustively prove that the correct input signal is selected and driven to `ResultY` for all eight selection codes.

8 L3 3 1

6. a) Write a Verilog module named `async_reset_dff` using behavioral modeling to implement a single D flip-flop with an asynchronous, active-high reset. The module must include inputs: `Clock`, `DataInput`, and `ResetAsync`. It should produce a state output `Q`. The key requirement is that the output `Q` is updated to the value of `DataInput` only on the positive edge of `Clock`, but `Q` is forced to '0' immediately whenever `ResetAsync` is asserted, irrespective of the clock state.

8 L3 4 1

- b) Provide a detailed discussion of the fundamental working principles of Volatile Memory (RAM), specifically describing the processes involved in initializing, accessing (reading), and altering (writing) data within a standard synchronous memory block. Following the discussion, develop a Verilog module named `sync_data_storage_16x8` that models a 16-word by 8-bit RAM using behavioral modeling. The module must include the following interface: `ClkInput`, `SelectAddress[3:0]`, `IncomingData[7:0]`, `WriteControl` (high for write, low for read), and `OutgoingData[7:0]`. Ensure the module:

i. Uses a memory declaration (e.g., `reg [7:0] memory [0:15];`).

ii. Performs a write operation (loading `IncomingData` into the addressed location) only when `WriteControl` is high, synchronized to the clock.

iii. Performs a read operation (outputting the data from the addressed location) when `WriteControl` is low, synchronized to the clock.

8 L3 4 1

Unit – III

7. a)

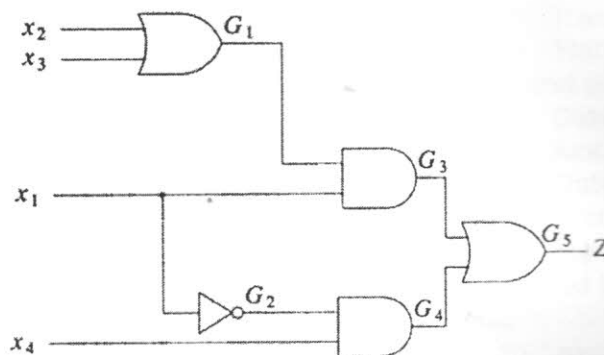


Fig. 7(a)

A logical circuit is shown in Fig. 7(a).

Using Path Sensitization method determine the Test Vector to check:

- | | | | | | |
|-------|--|----|----|---|---|
| i. | Stuck-at-0 fault in the path x2-G1-G3-G5-Z | 10 | L3 | 5 | 1 |
| ii. | Stuck-at-1 fault in the path x3-G1-G3-G5-Z | | | | |
| iii. | Stuck-at-0 fault in the path x1-G2-G4-G5-Z | | | | |
| iv. | Stuck-at-1 fault in the path x4-G4-G5-Z | | | | |
| b) | Sketch a sequence detector using the Moore FSM concept to detect the sequence "1010" (overlapping type). | 6 | L3 | 5 | 1 |
| 8. a) | Sketch a sequence detector using the Mealy FSM concept to detect the sequence "1010" (overlapping type). | 6 | L3 | 5 | 1 |
| b) | Write Verilog code for the sequence detector mentioned in Q.8(a). | 10 | L3 | 5 | 1 |

BT* Bloom's Taxonomy, L* Level; CO* Course Outcome; PO* Program Outcome
